สรุปคำสั่งและสถาปัตยกรรมโค้ดดิ้งสำหรับเตรียมสอบป้องกันโครงงาน

คู่มืออ้างอิงเชิงลึก (พฤศจิกายน 2569): เวิร์กโฟลว์การเชื่อมโยงระบบจริง SymPy, Streamlit และ Matplotlib

ภาควิชาวิทยาการคำนวณและเทคโนโลยีดิจิทัล คณะศิลปศาสตร์และวิทยาศาสตร์ มหาวิทยาลัยเกษตรศาสตร์

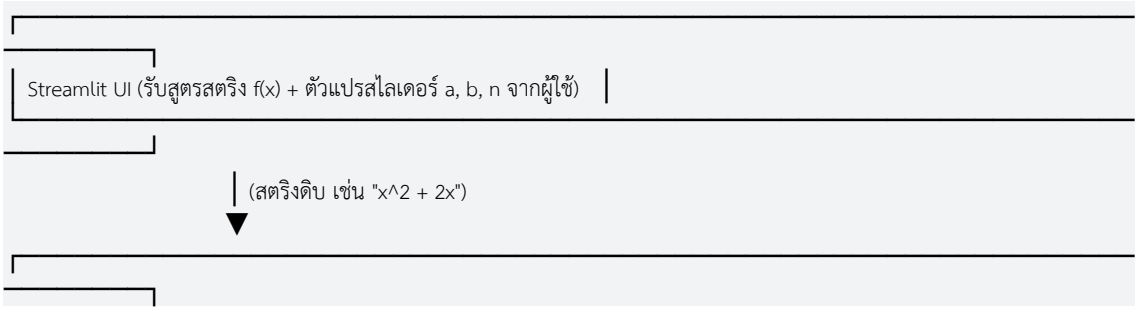
บทนำและภาพรวมสถาปัตยกรรมระบบ (System Architecture & Pipeline)

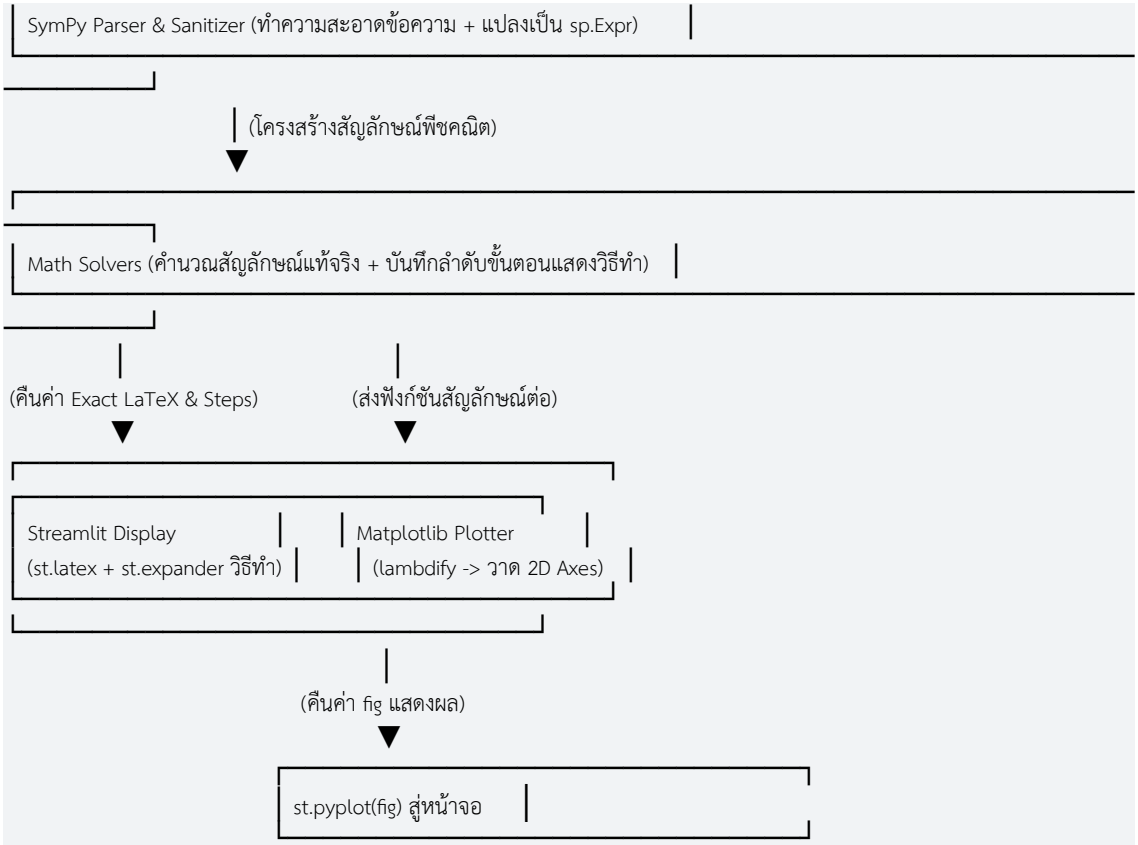
เอกสารฉบับนี้จัดทำขึ้นเพื่อให้นิสิตใช้ทบทวนโครงสร้างการเขียนโปรแกรม การทำงานของไลบรารีหลัก และเวิร์กโฟลว์การไหลของข้อมูล (Data Flow) ในโครงงาน **Calculus Learning Sandbox** เพื่อให้สามารถตอบคำถามของคณะกรรมการได้อย่างมั่นใจ เข้าใจกลไกการทำงานตั้งแต่ระดับผู้ใช้ป้อนข้อมูลไปจนถึงการประมวลผลและการแสดงผลลัพธ์

สถาปัตยกรรมสามชั้นและการไหลของข้อมูล (Three-Tier Data Pipeline)

การทำงานของระบบแบ่งออกเป็น 4 ขั้นตอนต่อเนื่องกันอย่างไร้รอยต่อ

1. **ส่วนติดต่อผู้ใช้งาน (Frontend Layer - Streamlit):**
    - รับข้อความสูตรคณิตศาสตร์จากผู้ใช้ผ่าน st.text_input และรับค่าตัวแปรควบคุม (เช่น ค่า a, b, n หรือ h) ผ่าน st.slider
    - ตรวจสอบความปลอดภัยเบื้องต้นด้วย sanitize_math_input() เพื่อป้องกันการป้อนข้อความที่ไม่พึงประสงค์ (RCE and DoS Protection)
2. **ส่วนคำนวณพีชคณิตสัญลักษณ์ (CAS Backend Layer - SymPy):**
    - แปลงข้อความสตริงเป็นโครงสร้างต้นไม้คณิตศาสตร์ (Abstract Syntax Tree) ผ่าน parse_expr() ร่วมกับกฎการคูณโดยนัย (implicit_multiplication_application)
    - ส่งต่อไปยัง Solver เฉพาะทาง (เช่น solve_riemann, solve_tangent, solve_substitution) เพื่อหาคำตอบที่แท้จริง (Exact Symbolic Form) และสร้างลำดับขั้นตอนวิธีทำ (Step-by-Step Derivation)
3. **ส่วนเรนเดอร์ภาพเรขาคณิต (Visualization Layer - Matplotlib):**
    - แปลงฟังก์ชันสัญลักษณ์ของ SymPy เป็นฟังก์ชันตัวเลขที่รวดเร็วด้วย sp.lambdify(..., modules=['numpy'])
    - สุ่มจุดพิกัดด้วย np.linspace() และวาดกราฟเส้นโค้ง แรเงาพื้นที่ หรือแท่งสี่เหลี่ยมรีมันน์ในโหมด Object-Oriented (Figure, Axes) ผ่านแบ็กเอนด์แบบไม่แสดงผลหน้าต่าง (Agg)
4. **ส่วนส่งคืนและนำเสนอผลลัพธ์ (Delivery & Rendering Layer):**
    - แสดงสมการคณิตศาสตร์ที่จัดรูปแล้วในรูป KaTeX ผ่าน st.latex()
    - แสดงกราฟิกอินเทอร์แอ็กทีฟผ่าน st.pyplot(fig)
    - แจ้งเตือนเงื่อนไขขอบเขตหรือข้อจำกัดทางคณิตศาสตร์ (Fail-safe Notices) ผ่าน st.warning() หรือ st.error()





ข้อตกลงข้อมูลผลลัพธ์มาตรฐาน (Solver Data Contract)

Solver ทุกตัวในระบบจะคืนค่าผลลัพธ์ในรูปแบบพจนานุกรม (Dictionary) ที่มีโครงสร้างมาตรฐานเดียวกันเสมอ เพื่อให้ส่วนติดต่อผู้ใช้สามารถนำไปแสดงผลได้อย่างปลอดภัยและเป็นระบบ

```
{
    "ok": True,                  # สถานะความสำเร็จ (True เมื่อคำนวณได้, False เมื่อเกิดข้อผิดพลาด)
    "result": sp_result,         # ออบเจกต์ผลลัพธ์สัญลักษณ์ของ SymPy (เช่น sp.Rational(4, 3))
    "latex": r"\frac{4}{3}",     # สตริงสมการ LaTeX สำหรับส่งให้ st.latex เรนเดอร์
    "exact_latex": r"\frac{4}{3}", # สตริง LaTeX สำหรับแสดงค่าแม่นยำแท้จริง
    "steps": [                   # รายการขั้นตอนการคำนวณทีละขั้นสำหรับแสดงใน Expander
        r"ขั้นที่ 1: กำหนดฟังก์ชัน f(x) = x^2",
        r"ขั้นที่ 2: หาปฏิยานุพันธ์ได้ F(x) = \frac{x^3}{3}",
        r"ขั้นที่ 3: แทนค่าขอบเขต F(2) - F(0) = \frac{8}{3}"
    ],
    "error": None,               # ข้อความแจ้งเตือนข้อผิดพลาด (มีค่าเป็นสตริงเมื่อ ok=False)
    "plot_data": {               # ข้อมูลตัวเลขประกอบสำหรับนำไปพล็อตกราฟ
        "f_num": callable_func,
        "x_range": (0.0, 2.0),
        "critical_points": [(2.0, 4.0)]
    }
}
```

สรุปคำสั่งใช้งาน SymPy เชิงลึก (SymPy CAS Cheatsheet)

SymPy ทำหน้าที่เป็นสมองกลหลักในการคำนวณพีชคณิตสัญลักษณ์ (Computer Algebra System) จุดเด่นคือการคงรูปค่าที่แม่นยำแท้จริง ไม่ปัดทศนิยม และตรวจสอบความถูกต้องทางตรรกะได้สมบูรณ์

1. การกำหนดตัวแปรสัญลักษณ์และการแปลงสตริงเป็นนิพจน์ (Parsing & Sanitization)

```
import sympy as sp
from sympy.parsing.sympy_parser import (
    parse_expr,
    standard_transformations,
    implicit_multiplication_application,
    convert_xor
)

# กำหนดชุดการแปลง: อนุญาตให้พิมพ์ 2x แทน 2*x และใช้ ^ แทนกำลังสอง
TRANSFORMATIONS = standard_transformations + (
    implicit_multiplication_application,
    convert_xor
)

# พจนานุกรมค่าคงตัวและฟังก์ชันพิเศษที่ผู้ใช้มักพิมพ์บ่อย
LOCAL_MATH_DICT = {
    "e": sp.E,
    "E": sp.E,
    "pi": sp.pi,
    "ln": sp.log
}

x = sp.Symbol('x', real=True)

# ฟังก์ชันแปลงข้อความผู้ใช้เป็นสัญลักษณ์อย่างปลอดภัย
raw_input = "x^2 + 2x - cos(x)"
expr = parse_expr(raw_input, transformations=TRANSFORMATIONS, local_dict=LOCAL_MATH_DICT)
```

2. การคำนวณลิมิตและการตรวจสอบการหาค่าได้ (Limits & DNE Handling)

```
# ลิมิตสองด้าน ลิมิตซ้าย และลิมิตขวา
f = (x**2 - 4) / (x - 2)

L_both  = sp.limit(f, x, 2)          # ได้ 4
L_left  = sp.limit(f, x, 2, dir='-')  # ได้ 4
L_right = sp.limit(f, x, 2, dir='+')  # ได้ 4

# การตรวจสอบว่าลิมิตหาค่าไม่ได้ (Does Not Exist - DNE)
abs_f = sp.Abs(x) / x
L_sub_left  = sp.limit(abs_f, x, 0, dir='-')  # ได้ -1
L_sub_right = sp.limit(abs_f, x, 0, dir='+')  # ได้ +1
```

```
if L_sub_left != L_sub_right:
    status = "ลิมิตหาค่าไม่ได้ (DNE) เนื่องจากลิมิตซ้ายไม่เท่ากับลิมิตขวา"
```

3. การหาอนุพันธ์และสมการเส้นสัมผัส (Derivatives & Tangent Lines)

```
f = x**2
a_val = 1

# คำนวณอนุพันธ์ f'(x) และความชัน ณ จุด x = a
f_prime = sp.diff(f, x)
slope_m = f_prime.subs(x, a_val)              # ความชัน m = 2
y_val = f.subs(x, a_val)                      # พิกัด y = 1

# หาสมการเส้นสัมผัส y - y0 = m(x - x0)
y_tangent = slope_m * (x - a_val) + y_val     # y = 2x - 1
```

4. การอินทิเกรตจำกัดเขตและการแปลงสัญลักษณ์ LaTeX (Integration & LaTeX)

```
f = x**2

# ปริพันธ์ไม่จำกัดเขต (Indefinite Integral)
indef = sp.integrate(f, x)                    # x**3 / 3

# ปริพันธ์จำกัดเขต (Definite Integral บนช่วง [0, 2])
def_val = sp.integrate(f, (x, 0, 2))          # 8 / 3 (คงรูป sp.Rational)

# การแปลงเป็นโค้ด LaTeX สำหรับนำไปแสดงผล
latex_expr = sp.latex(f)                      # "x^{2}"
latex_ans  = sp.latex(def_val)                # "\frac{8}{3}"
```

5. การดักจับข้อยกเว้นและกรณีพิเศษ (Fail-safe & Exception Traps)

```
# การดักจับจุดเอกฐาน อนันต์ และฟังก์ชันที่ไม่มีปฏิยานุพันธ์มูลฐาน
try:
    res = sp.integrate(sp.exp(-x**2), x)
    # หากผลลัพธ์ติดฟังก์ชันพิเศษ erf(x) ที่ไม่ใช่ฟังก์ชันมูลฐาน
    if res.has(sp.erf):
        print("ฟังก์ชันนี้ไม่มีปฏิยานุพันธ์ในรูปฟังก์ชันมูลฐาน")
except (sp.PoleError, ValueError, NotImplementedError) as e:
    print(f"เกิดข้อผิดพลาดทางคณิตศาสตร์: {e}")

# ตรวจสอบค่าผลลัพธ์ที่เป็นอนันต์หรือไม่นิยาม
if def_val in (sp.oo, -sp.oo, sp.zoo, sp.nan):
    print("ปริพันธ์มีค่าลู่ออก (Divergent)")
```

สรุปคำสั่งใช้งาน Streamlit เชิงลึก (Streamlit UI Cheatsheet)

Streamlit ทำหน้าที่จัดการหน้าจอ วิดเจ็ตนำเข้าข้อมูล และผังการแสดงผลแบบหลายหน้า (Multipage Architecture)

1. โครงสร้างโฟลเดอร์แบบหลายหน้า (Multipage File Structure)

```
calculus-integration-app/
├── app.py                  # Entry point หลัก ควบคุม Sidebar Navigation
├── pages/
│   ├── home.py             # หน้าแรกและแนะนำโครงงาน
│   ├── lessons.py          # ศูนย์รวมบทเรียน 7 โมดูล
│   ├── riemann.py          # โมดูลผลรวมรีมันน์อินเทอร์แอ็กทีฟ
│   ├── tangent.py          # โมดูลเส้นสัมผัสและอัตราการเปลี่ยนแปลง
│   ├── substitution.py     # โมดูลเทคนิคการแทนค่า u
│   ├── area_between.py     # โมดูลพื้นที่ระหว่างสองเส้นโค้ง
│   ├── volume_revolution.py  # โมดูลปริมาตรทรงตันจากการหมุน
│   ├── improper_integrals.py  # โมดูลปริพันธ์ไม่ตรงแบบ
│   ├── solver.py           # เครื่องมือคำนวณแคลคูลัสอิสระ
│   └── quiz.py             # ระบบทดสอบวัดมโนทัศน์ 80 ข้อ
└── utils/                  # โมดูลคำนวณและวาดกราฟเบื้องหลัง
```

2. การตั้งค่าหน้าจอและโครงสร้างเลย์เอาต์ (Page Configuration & Columns)

```python
import streamlit as st


# ตั้งค่าหน้าจอเป็นแบบเต็มความกว้าง (Wide Mode)
st.set_page_config(
    page_title="Calculus Learning Sandbox",
    page_icon="📐",
    layout="wide",
    initial_sidebar_state="expanded"
)


# การแบ่งพื้นที่เป็นคอลัมน์ซ้าย-ขวา (เช่น ฝั่งซ้ายป้อนข้อมูล ฝั่งขวาแสดงกราฟ)
col_input, col_display = st.columns([1, 1.2], gap="medium")


with col_input:
    st.subheader("พารามิเตอร์ควบคุม")
    expr_text = st.text_input("ป้อนสูตรฟังก์ชัน f(x):", value="x^2")
    n_slices = st.slider("จำนวนแท่งสี่เหลี่ยม (n):", min_value=1, max_value=100, value=10)


with col_display:
    st.subheader("การแสดงผลเรขาคณิต")
    # พื้นที่สำหรับใส่ st.pyplot(fig)
```

3. การแสดงผลสมการคณิตศาสตร์และกล่องวิธีทำ (KaTeX & Expanders)

```python
# แสดงสมการคณิตศาสตร์แบบเด่นชัดตรงกลาง
st.latex(r"\int_{a}^{b} f(x)\,dx = \lim_{n \to \infty} \sum_{i=1}^{n} f(x_i^*)\,\Delta x")
```

```
# การทำกล่องวิธีทำแบบยืดหดได้ เพื่อไม่ให้รกสายตา (Scaffolding Level 2)
with st.expander("🔍 ดูขั้นตอนการคำนวณอย่างละเอียด (Step-by-Step)", expanded=False):
    for step in steps_list:
        st.write(step)
```

4. การจัดการสถานะและความจำเซสชัน (Session State Management)

```
# บันทึกสถานะคะแนนและข้อสอบปัจจุบันในระบบควิซ
if "quiz_score" not in st.session_state:
    st.session_state.quiz_score = 0
if "current_q_index" not in st.session_state:
    st.session_state.current_q_index = 0


def on_submit_answer(is_correct: bool):
    if is_correct:
        st.session_state.quiz_score += 1
        st.toast("🎉 ตอบถูกต้อง!", icon="✅")
    else:
        st.toast("คำตอบยังไม่ถูกต้อง ลองดูคำใบ้อีกครั้งนะคะ", icon="💡")
```

สรุปคำสั่งใช้งาน Matplotlib เชิงลึก (Matplotlib Plotter Cheatsheet)

Matplotlib ทำหน้าที่แปลงมโนทัศน์เชิงพีชคณิตให้กลายเป็นภาพกราฟิก 2 มิติที่ชัดเจน ถูกต้อง และสะอาดตา

1. การสร้าง Figure แบบป้องกัน Memory Leak (Object-Oriented Pattern)

```
import matplotlib
matplotlib.use("Agg")  # ใช้แบ็กเอนด์แบบไม่มีหน้าต่าง ป้องกัน GUI crash บนคลาวด์
from matplotlib.figure import Figure
import matplotlib.pyplot as plt
import numpy as np


# สร้าง Figure และ Axes โดยตรง ไม่ใช้ plt.subplots() เพื่อลดการค้างของ RAM
fig = Figure(figsize=(8, 4.8), dpi=100)
ax = fig.add_subplot(111)


# จุดสำคัญมาก: เมื่อส่ง fig ให้ st.pyplot แล้ว ต้องปิด figure เสมอ
# st.pyplot(fig)
# plt.close(fig)
```

2. การเซ็ตแกนพิกัดฉากตัดศูนย์ (Spines Center Origin)

```
# เลื่อนแกน x และ y ให้ตัดกันที่จุดกำเนิด (0, 0) แบบคณิตศาสตร์แท้จริง
ax.spines['left'].set_position('zero')
ax.spines['bottom'].set_position('zero')
ax.spines['right'].set_color('none')
ax.spines['top'].set_color('none')
```

```
# กำหนดเส้นตารางที่ดูนุ่มนวลสบายตา
ax.grid(True, linestyle="--", alpha=0.5, color="#CBD5E1")
```

3. การพล็อตเส้นโค้งและจุดโปร่งลิมิต (Curves & Open Holes)

```
xs = np.linspace(-3, 5, 400)
ys = xs + 2  # สมมติกราฟ f(x) = (x^2-4)/(x-2)


# พล็อตเส้นโค้งหลัก
ax.plot(xs, ys, color="#1E40AF", linewidth=2.5, label="f(x)")


# พล็อตจุดโปร่ง (Hole) ณ จุด x = 2 ที่ฟังก์ชันไม่นิยาม
# ใช้มาร์กเกอร์รูปวงกลม 'o' กำหนด markerfacecolor='none' และสีกรอบเข้ม
ax.plot([2], [4], marker='o', markersize=8, markerfacecolor='white',
        markeredgecolor="#DC2626", markeredgewidth=2, zorder=5, label="จุดไม่นิยาม (2, 4)")
```

4. การแรเงาพื้นที่ระหว่างเส้นโค้งและการวาดสี่เหลี่ยมรีมันน์ (Shading & Bars)

```
from matplotlib.patches import Rectangle


# การแรเงาพื้นที่ใต้กราฟบนช่วง [a, b]
x_sub = np.linspace(0, 2, 200)
y_sub = x_sub**2
ax.fill_between(x_sub, y_sub, 0, color="#FBCFE8", alpha=0.6, label="พื้นที่ปิดล้อม")


# การวาดแท่งสี่เหลี่ยมผืนผ้าผลรวมรีมันน์
delta_x = 0.5
x_bar = 0.0
height = x_bar**2


rect = Rectangle(
    (x_bar, 0), delta_x, height,
    facecolor="#FDE047", edgecolor="#18181B", linewidth=1.2, alpha=0.7
)
ax.add_patch(rect)
```

เวิร์กโฟลว์เชื่อมโยง 3 ไลบรารีสมบูรณ์ (End-to-End Mini App Workflow)

ตัวอย่างโค้ดด้านล่างนี้จำลองโครงสร้างย่อของระบบจริงในไฟล์เดียว แสดงให้เห็นการเชื่อมโยงข้อมูลตั้งแต่การรับ Input จาก Streamlit → คำนวณใน SymPy → พล็อตภาพใน Matplotlib → และเรนเดอร์กลับสู่หน้าจออย่างเป็นระบบ

```
import streamlit as st
import sympy as sp
import numpy as np
import matplotlib
matplotlib.use("Agg")
from matplotlib.figure import Figure
import matplotlib.pyplot as plt
```

```python
def run_calculus_pipeline():
    st.title("Calculus Pipeline Minimal Demo")

    # 1. Frontend Layer: รับค่าจากผู้ใช้
    user_func = st.text_input("ระบุฟังก์ชัน f(x):", value="x^2")
    col1, col2 = st.columns(2)
    with col1:
        a_val = st.number_input("ขอบเขตล่าง (a):", value=0.0)
    with col2:
        b_val = st.number_input("ขอบเขตบน (b):", value=2.0)

    # 2. CAS Layer: ประมวลผลด้วย SymPy
    x = sp.Symbol('x', real=True)
    try:
        expr = sp.sympify(user_func)
        # คำนวณปริพันธ์จำกัดเขตที่แท้จริง
        exact_integral = sp.integrate(expr, (x, a_val, b_val))
        latex_str = sp.latex(exact_integral)

        # แสดงผลลัพธ์สมการพีชคณิตแท้จริง
        st.success(f"ผลการอินทิเกรตที่แท้จริง:")
        st.latex(rf"\int_{{{a_val}}}^{{{b_val}}} ({sp.latex(expr)})\,dx = {latex_str}")

        # 3. Visualization Layer: วาดกราฟเรขาคณิตด้วย Matplotlib
        # แปลงฟังก์ชันสัญลักษณ์เป็นฟังก์ชันตัวเลข NumPy
        f_num = sp.lambdify(x, expr, modules=["numpy"])
        xs = np.linspace(a_val - 1.0, b_val + 1.0, 300)
        ys = f_num(xs)

        fig = Figure(figsize=(7, 3.8))
        ax = fig.add_subplot(111)

        # จัดแกนพิกัดฉาก
        ax.spines['left'].set_position('zero')
        ax.spines['bottom'].set_position('zero')
        ax.spines['right'].set_color('none')
        ax.spines['top'].set_color('none')
        ax.grid(True, linestyle=":", alpha=0.6)

        # พล็อตเส้นโค้งและแรเงาพื้นที่
        ax.plot(xs, ys, color="#2563EB", linewidth=2.2, label=f"y = {user_func}")
        xs_fill = np.linspace(a_val, b_val, 200)
        ys_fill = f_num(xs_fill)
        ax.fill_between(xs_fill, ys_fill, 0, color="#BFDBFE", alpha=0.5, label="Area")
        ax.legend(loc="upper left")

        # 4. Delivery Layer: แสดงภาพและเคลียร์หน่วยความจำ
        st.pyplot(fig)
        plt.close(fig)

    except Exception as err:
        st.error(f"ไม่สามารถคำนวณฟังก์ชันได้: {err}")
```